Alexander Abuabara
DAEN 489 - Spring 2026
College Station 22-Jan-2026

# Week 2

**(part 1 💬)**

## Contents

1 **Package sf** 1

2 **Attribute Data Operations** 9

# 1 Package sf

- R package `sf` provides a table format for simple features, where feature geometries are stored in a list-column.
- R package `stars` was written to support raster and vector data cubes (we will see that later), supporting raster layers, raster stacks and feature time series as special cases.
- `sf` first appeared on CRAN in 2016, `stars` in 2018. Development of both packages received support from the R Consortium as well as strong community engagement.
- The packages were designed to work together. Functions or methods operating on `sf` or `stars` objects start with `st_`, making it easy to recognize them or to search for them when using command line completion.

```r
library(sf)

methods(class = "sf")
```

### Creation

An sf object can be created from scratch by

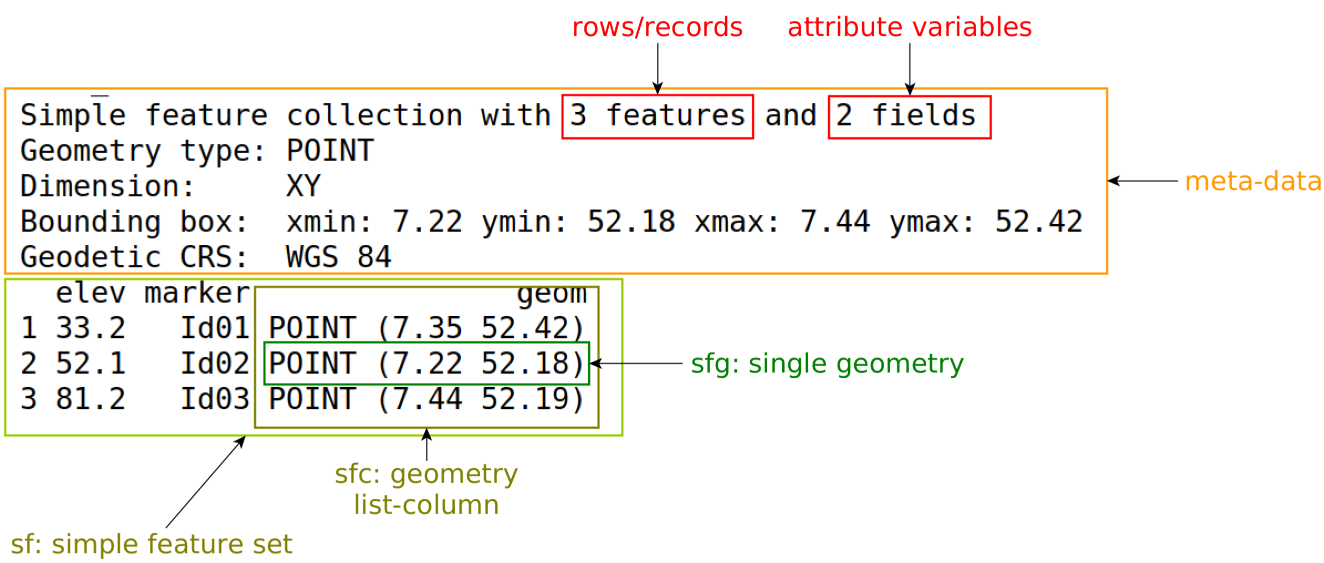
```r
library(sf)

p1 <- st_point(c(7.35, 52.42))
p2 <- st_point(c(7.22, 52.18))
p3 <- st_point(c(7.44, 52.19))

sfc <- st_sfc(list(p1, p2, p3), crs = 'OGC:CRS84')

st_sf(elev = c(33.2, 52.1, 81.2),
      marker = c("Id01", "Id02", "Id03"), geom = sfc)

plot(sfc)
```




## Reading and writing

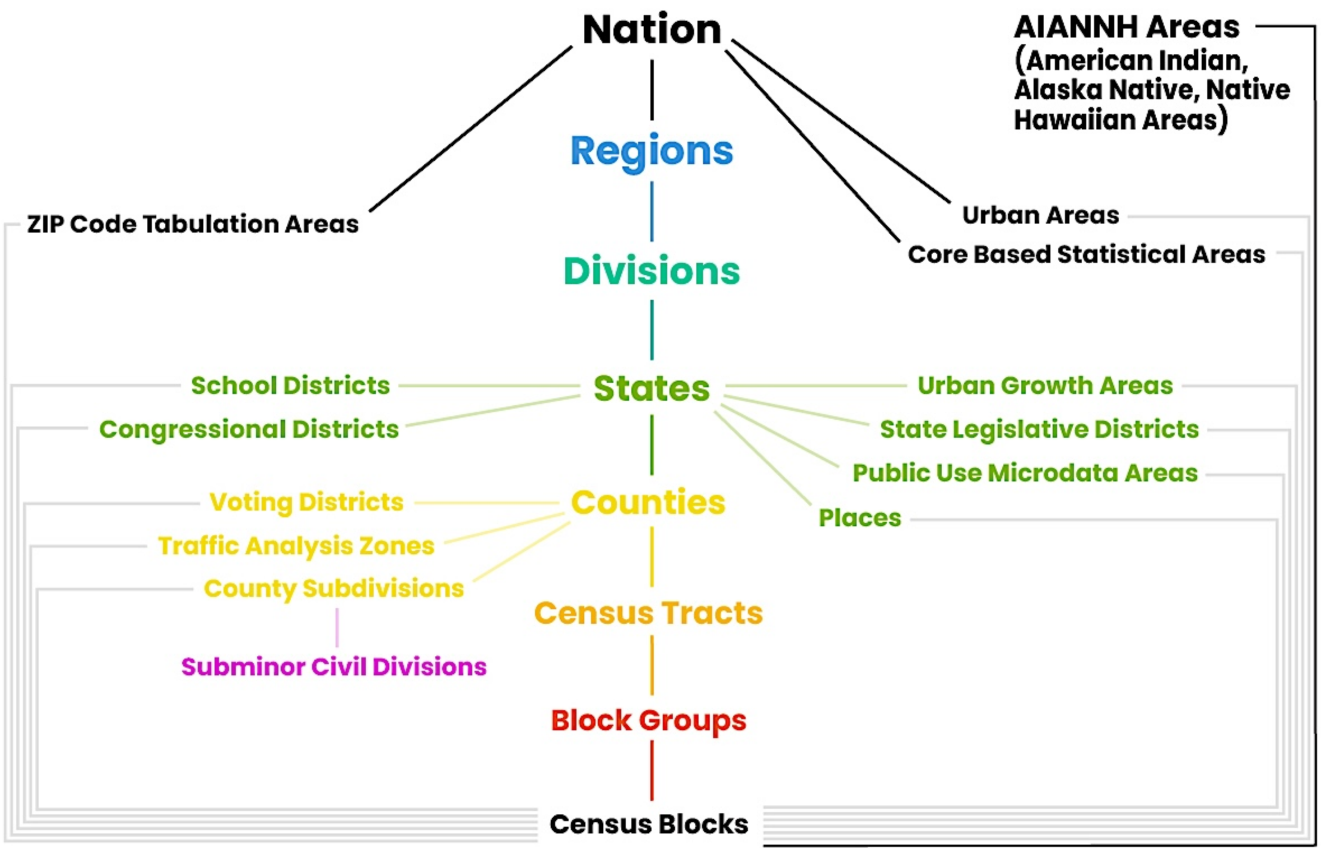
```r
library(sf)

(file <- system.file("gpkg/nc.gpkg", package = "sf"))

nc <- st_read(file)

st_layers(file)
```

### US Census TIGER/Line Shapefiles




### TIGER=Topologically Integrated Geographic Encoding & Referencing

- Nationwide spatial database maintained by the U.S. Census Bureau
- Administrative boundaries: states, counties, tracts, block groups, blocks
- Legal and statistical areas: places, ZCTAs
- Features: roads, rail, hydrography, landmarks
  - Official geometry for Decennial Census and ACS products
  - Ensures consistent geographies across datasets and time
  - Free, open, and nationally comprehensive

### Typical uses

- Joining tabular census data to spatial boundaries
- Choropleth mapping and spatial aggregation
- Defining analysis units for demographic and policy studies

### Key characteristics

- Updated annually, standardized schema, nationwide coverage
- Designed for analysis, not high-precision surveying

### Common Pitfalls When Using TIGER/Line Data

- **Geographic units are not stable over time**
  - Tracts, block groups, and boundaries change between censuses
  - Always align year of geometry with year of attributes
- **ZCTAs are not ZIP codes**
  - ZCTAs approximate USPS ZIPs, but are not identical
  - Not appropriate for all mailing or service-area analyses
- **Projection and coordinate system issues**
  - Shapefiles are typically in geographic coordinates (NAD83)
  - Reproject before distance, area, or buffering operations
- **Geometry quality limitations**
  - Boundaries are generalized for national coverage
  - Not suitable for parcel-level or engineering-grade analysis
- **Joining errors are common**
  - Mismatched GEOIDs, leading zeros, and type coercion
  - Always validate join completeness and key formats
- **Large file sizes and performance**
  - National layers can be very large
  - Subset by state or county for efficient workflows

### Hands-on

- Download: **Counties (and equivalent)**
  `https://www.census.gov/geographies/mapping-files/2025/geo/tiger-line-file.html`
- Unzip and copy path to "*tl_2025_us_county.shp*"

```
us_counties <-
  st_read("/Users/abuabara/.../tl_2025_us_county/tl_2025_us_county.shp")

class(us_counties)

dim(us_counties)
```

```r

glimpse(us_counties)

library(dplyr) # tidyverse

glimpse(us_counties)

filter(us_counties, STATEFP == 48)

tx_counties <- filter(us_counties, STATEFP == 48)

# plot(tx_counties)
# plot(st_geometry(tx_counties))
```

## Very Basic Plotting

Ref.:https://r-spatial.github.io/sf/articles/sf5.html

More details about the map's composition will be explored later (Week 9).

### ggplot2

- `ggplot2` is a powerful R package for creating **statistical graphics** using a consistent, structured grammar.
- It is based on the **Grammar of Graphics**, which decomposes every plot into a small set of independent components.
- Instead of choosing a plot type first, you:
  - Specify the data,
  - Map variables to visual features, and
  - Add layers that build the final figure.
- The basic template is:

```
ggplot(data = ..., mapping = aes(...)) + geom_...
```

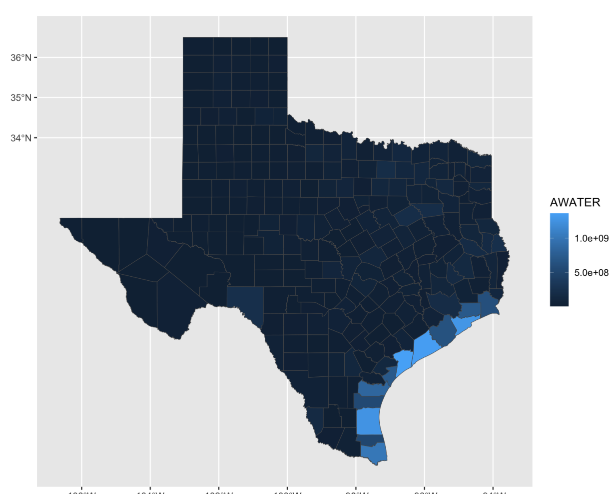
- Every ggplot has three core parts:
  - **Data**: the data frame being plotted,
  - **Aesthetics** (`aes`): how variables map to x, y, color, size, etc.,
  - **Geoms**: the geometric objects that draw the plot (points, lines, …).

- Plots are built **incrementally** using the `+` operator:
  - Start with a base plot,
  - Then add layers, scales, labels, and themes.
- The same data can produce many different plots by changing only the **geom**.
- `ggplot2` encourages:
  - Reproducibility,
  - Clear separation between data and presentation, and
  - Code that reads like a description of the graphic.
- Common first geoms:
  - `geom_point()` for scatterplots,
  - `geom_line()` for time series,
  - `geom_bar()` for counts and categories.
- By default, `ggplot2` produces **publication-quality** figures with sensible styling.

```r
install.packages("ggplot2")
```

```r
library(ggplot2)

ggplot() +
  geom_sf(data = tx_counties, aes(fill = AWATER)) +
  scale_y_continuous(breaks = 34:36)
```



### tmap

**tmap** is powerful for creating thematic maps with "elegant", flexible syntax.

- Layer-based approach similar to ggplot2
- Supports both static and interactive maps
- Works seamlessly with sf (simple features) objects
- Publication-ready cartographic output
- Easy to customize colors, legends, and layouts

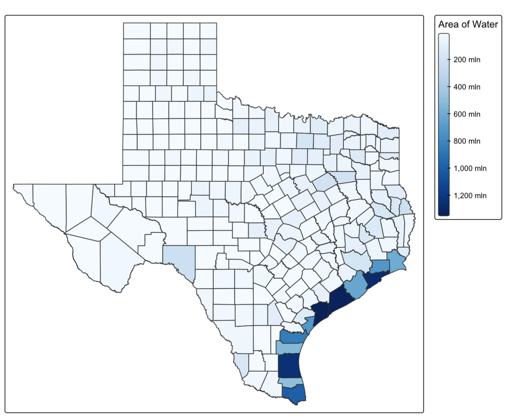
```r
install.packages("tmap")
```

The tmap syntax follows a layered approach:

```r
library(tmap)

# version 3
tm_shape(tx_counties) +
  tm_polygons("AWATER",
              palette = "Blues",
              title = "Area of Water")

# version 4
tm_shape(tx_counties) +
  tm_polygons(fill = "AWATER",
              fill.scale = tm_scale_continuous(values = "brewer.blues"),
              fill.legend = tm_legend(title = "Area of Water"))
```



## Common functions:

- `tm_shape()`: Define the spatial data
- `tm_polygons()`: Draw filled polygons
- `tm_borders()`: Draw polygon boundaries
- `tm_dots()`: Add point symbols
- `tm_text()`: Add labels

Some interesting options to explore:

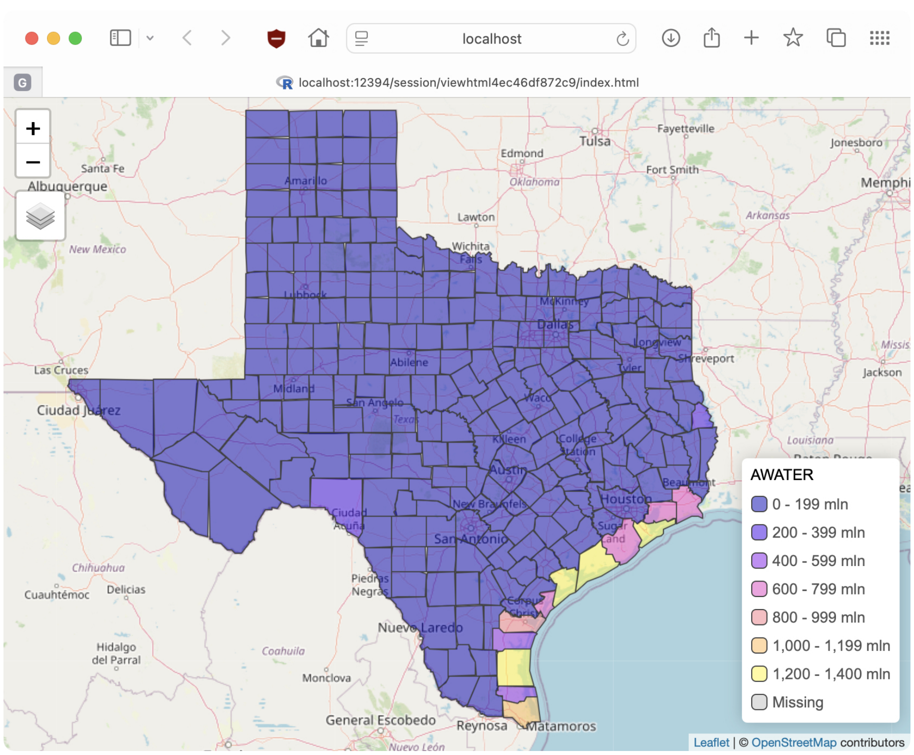
- Draw a thematic map quickly (quick thematic map plot)

```r
qtm(tx_counties)
```

- This function is a convenient wrapper of the main plotting method of stacking tmap-elements.
- Without arguments or with a search term, this functions draws an interactive map.
- Set tmap mode to static plotting or interactive viewing:

```r
tmap_mode("view")

tm_shape(tx_counties) + tm_polygons("AWATER", palette = sf.colors(5), alpha = .5)
```



# 2 Attribute Data Operations

- Attribute data operations focus on **selecting, creating, transforming, and summarizing columns** in a data frame.
- In R, attributes (columns) are commonly accessed using:
  - Column names: `df$variable`
  - Bracket indexing: `df[, "variable"]` or `df[, j]`
- Common goals of attribute operations:
  - Select a subset of variables,
  - Create new variables from existing ones,
  - Modify or recode variables,
  - Compute summaries by column.
- **Selecting and Subsetting Attributes**
  - Extract a single column: `df$age`, `df[, "age"]`
  - Extract multiple columns: `df[, c("age", "income", "gender")]`
  - Remove columns by index or name: `df[, -3]`, `df[, !names(df) %in% c("id", "timestamp")]`
- **Creating and Modifying Attributes**
  - Create a new attribute: `df$income_thousands <- df$income / 1000`
  - Modify an existing attribute: `df$age <- df$age + 1`
  - Vectorized operations apply to the entire column at once.
- **Recoding and Transforming Attributes**
  - Apply transformations: `log(df$income)`, `scale(df$score)`
  - Recode categorical variables: `df$gender <- factor(df$gender, levels = c("M", "F"))`
  - Create indicator variables: `df$is_adult <- df$age >= 18`
- **Summarizing Attributes**
  - Column-wise summaries: `mean(df$age, na.rm = TRUE)`, `summary(df)`
  - Apply a function to columns: `sapply(df, mean, na.rm = TRUE)`
  - Grouped summaries with `aggregate()` or `dplyr::summarise()`
- **Good Practices**: Inspect structure and names: `names(df)`, `str(df)`, `glimpse(df)`. Be mindful of missing values, data types, and recycling rules. Prefer readable code and avoid hard-coded column indices.

```r
us_counties_df <- st_drop_geometry(us_counties)

class(us_counties_df)

glimpse(us_counties_df)

dim(us_counties_df)
ncol(us_counties_df)
nrow(us_counties_df)
```

# Vector Attribute Manipulation

```r
tx_counties$AREA = tx_counties$ALAND + tx_counties$AWATER

st_crs(tx_counties)

tx_counties$AREA_km2 = tx_counties$AREA / 1e+6

tx_counties = mutate(tx_counties, Area2 = (ALAND + AWATER) / 1e+6)
```

## Vector Attribute Subsetting

A demonstration of the utility of using logical vectors for subsetting is shown in the code chunk below.

```r
tx_counties[3:4, ]        # subset rows by position
tx_counties[, 3:4]        # subset columns by position
tx_counties[, c(1,4,7)]   # subset columns by position
tx_counties[4:6, 3:6]     # subset rows and columns by position
tx_counties[, c("GEOID", "NAME")] # columns by name
tx_counties[, c(F, T)]    # by logical indices
tx_counties[, 888]        # an index representing a non-existent column
```

This creates a new object, small_counties, containing counties whose surface area is smaller than 1,000 km2.

```
i_small = tx_counties$AREA_km2 < 1000
summary(i_small) # a logical vector
small_tx_counties = tx_counties[i_small, ]
```

The intermediary i_small (short for index representing small counties) is a logical vector that can be used to subset the nine smallest counties in Texas by surface area. A more concise command, which omits the intermediary object, generates the same result:

```
small_tx_counties = tx_counties[tx_counties$AREA_km2 < 1000, ]
```

The base R function `subset()` provides another way to achieve the same result:

```
small_tx_counties = subset(tx_counties, AREA_km2 < 1000, )
```

Base R functions are mature, stable and widely used, making them a rock solid choice, especially in contexts where reproducibility and reliability are key.

`dplyr` functions enable 'tidy' workflows which a lot of people find intuitive and productive for interactive data analysis, especially when combined with code editors such as RStudio that enable auto-completion of column names.

Key functions for subsetting data frames (including `sf` data frames) with `dplyr` functions are demonstrated below.

```
small_tx_counties = filter(tx_counties, AREA_km2 < 1000)
```

NEXT WEEK:

### Chaining Commands with Pipes

### Vector Attribute Aggregation

### Vector Attribute Joining

### Creating Attributes and Removing Spatial Information

## Manipulating Raster Objects

### Raster Subsetting

### Summarizing Raster Objects

## Pre-processing and Workflows